# EXECUTION-PATH QUALIFICATION AND REALIZED COSTS IN SPECULATIVE DECODING ON CONSUMER SYSTEMS

**Anonymous Authors**[1]

## ABSTRACT

Fast drafting and cheap target batching need not yield faster speculative decoding once output preservation and state management are included. We characterize four consumer systems, retaining profitable, unprofitable, and blocked outcomes with campaign-specific comparators. Two fresh direct-ID studies show that serialization restores qualification on RTX (75/90 to 90/90) but not B570 (60/90 to 78/90). A controlled B570 follow-up freezes a target-partition hypothesis before evaluating a reserved workload; target-only micro-batch one predicts all 72 serialized output sequences, including the residual divergences. On the tested stack, this isolates a sufficient target execution partition. Direct timing shows that an isolated NPU draft advantage does not survive the tested 358H stateful interface; a later intervention stops at qualification. On 60 held-out M4 prompts, a one-way continuation guard achieves 1.3114× post-prefill speedup over target-only (95% prompt-bootstrap CI [1.2487, 1.3787]); Guard and fixed speculation preserve all 180/180 matched generated-token sequences. Fixed speculation achieves 1.3469×, and the paired secondary comparison finds Guard slower. The result establishes no incremental advantage of adding control.

## 1 INTRODUCTION

Speculative decoding proposes tokens with a cheaper draft for batched target validation (Leviathan et al., 2023). Consumer-hardware characterization already shows both gains and slowdowns when verification lacks effective parallel execution (Chordiya, 2026). We ask a preceding systems question: whether the physical execution path is admissible under the intended output contract, and whether its local opportunity survives complete realized cost.

We examine M4, RTX 5070 Ti Laptop, Arc B570, and 358H (Table 1). M4 representative and separately preregistered HumanEval workloads yield 0.842× and 1.387× speedup under historical text-path comparators. Session differences prevent a workload-only attribution.

Historical RTX and B570 controls first identified a verification-path qualification issue. Two subsequent prospectively frozen direct-ID studies share HumanEval30 task IDs, the Qwen3-8B/0.6B Q8_0 model pair, greedy $k = 2$, three repetitions, and a 90/90 qualification rule. RTX agreement changes from 75/90 batched pairs to 90/90 serialized pairs; B570 changes from 60/90 to 78/90 and remains blocked. A controlled B570 follow-up isolates target execution partition as a sufficient predictor of the serialized output trajectory: target-only micro-batch one matches 36/36 exposed discovery sequences, then predicts 72/72 complete sequences on a separately frozen reserved set. On 358H, the isolated NPU draft advantage does not survive the tested stateful integration, while complete target-side work dominates CPU drafting. All timing gains concern post-prefill decode, excluding loading and initial prompt prefill.

Our three contributions are (1) prospectively frozen direct-ID execution-path qualification on RTX/B570; (2) controlled B570 target-partition isolation with frozen 72/72 held-out sequence prediction; and (3) direct 358H stateful-cost characterization. Findings concern the measured configurations and campaign-specific comparators. M4 supplies runtime evidence, without demonstrated Guard benefit over fixed speculation. Heterogeneous placement, cost prediction, numerical path dependence, deterministic inference, and adaptive speculation are established (Ruiz y Mesa et al., 2026; Wang et al., 2026; Yuan et al., 2025; Gond et al., 2026; Liu et al., 2026a).

For practitioners, the evidence supports an evaluation sequence: establish path admissibility under the intended output contract; if it fails, isolate target execution partition before attributing failure to drafting or placement; then measure complete realized cost on a qualified path. This sequence follows the tested boundaries rather than defining a universal algorithm.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

## 2 Experimental Question and Measurement Discipline

We investigate the gap between favorable component measurements and integrated post-prefill decode outcomes. Acceptance describes candidate agreement, isolated draft throughput describes one execution boundary, and target-forward batching describes another. None alone measures the complete speculative loop. We use an output contract and an accounting lens to make those boundaries explicit before comparing the frozen campaigns.

### 2.1 A path-qualified output contract

A greedy speculative round proposes a draft sequence, verifies it with the target, and commits the matching prefix followed by a correction or bonus token when the remaining output budget permits. Rejected candidate state must be removed or reconstructed consistently. Our intended output contract requires generated token IDs to equal the matched target-only greedy output under the frozen configuration. This makes output preservation an explicit requirement and compares latency for the same generated sequence. It is an empirical property of a model, runtime, execution path, and workload, not a property inferred from the algorithm name.

The recorded comparators are not uniform. Historical M4, RTX, and B570 campaigns compare token arrays reconstructed from parsed CLI text; their reported equality counts establish agreement through that text-processing path, not direct generated-ID equality. The 358H runner, prospective M4 runtime study, and new separately frozen RTX/B570 qualification studies compare generated IDs directly. We retain the historical counts and evidence classes, but do not promote text-path agreement to proof of the stricter ID contract.

We call a path *admissible* only within its tested workload and recorded comparator. For historical text-path comparisons this denotes campaign qualification, not certification under the direct-ID contract. A failed pair blocks a workload-global optimization claim even if the timing is favorable. A small qualification probe establishes eligibility for broader testing, not universal equivalence. We retain blocked-path timings as diagnostics with their correctness status visible. This deliberate contract is stricter than an approximate-quality deployment objective; HumanEval token equality does not establish functional correctness or pass@$k$.

### 2.2 Evidence classes

**Primary** results answer a frozen workload question after its correctness gate. **Secondary** results are registered variants or explicitly bounded follow-up controls. **Mechanism-supported** claims use path controls or timed regions. **Diagnostic** values describe correctness-blocked or otherwise restricted paths. These labels describe permitted inference, not just measurement precision. In particular, the text-path-qualified RTX serialized controls are secondary evidence; they do not replace its blocked primary fast path. Historical B570 follow-ups remain diagnostic after the formal stop; the new qualification studies have a separate prospective direct-ID endpoint.

### 2.3 Complete round costs

Let $T_b$ be the matched target-only cost per output token. The realized draft time $T_d^{\text{real}}$ denotes the realized draft-subsystem time per round, including any prefix replay performed by its interface. The complete target time $T_{\text{target,round}}^{\text{real}}$ includes all measured target-side round work: candidate evaluation and any correction, bonus, or rollback work inside that region. Residual synchronization, transfer, orchestration, and bookkeeping time not included in either region is $T_x$. Let $C$ count *all* committed output tokens. With consistent timing and aggregation boundaries,

$$S = \frac{T_b C}{T_d^{\text{real}} + T_{\text{target,round}}^{\text{real}} + T_x} \tag{1}$$

$$= \frac{C}{D_{\text{real}} + R_{\text{target}} + X}, \tag{2}$$

where each denominator term is normalized by $T_b$. Break-even requires $C > D_{\text{real}} + R_{\text{target}} + X$. These identities organize measurements; they are not a new profitability theory. Cost/commit decompositions already appear in runtime-controlled studies (Kim et al., 2026).

At an aggregate level, round means and matched total-time/token accounting must use the same weights. A ratio of total times need not equal a geometric mean of prompt speedups. Likewise, accepted draft tokens per round omit correction or bonus output and cannot automatically be used as $C$. We do not infer unmeasured replay-only, transfer-only, or correction-only components by subtraction from incompatible experiments.

### 2.4 A separate target-forward proxy

For context $n$ and path $e$, define

$$V_k^{\text{micro}}(n, e) = \frac{T_{\text{target forward}}(k, n, e)}{T_{\text{target decode}}(1, n)}. \tag{3}$$

This microbenchmark asks whether a target can process a causal block near the cost of a single decode. It omits draft generation and can omit target correction/bonus and state-management work. Consequently $V_k^{\text{micro}}$ is not automatically $R_{\text{target}}$. Likewise, isolated draft throughput cannot substitute for realized stateful draft cost. These distinctions guide diagnosis after the observed outcomes; they do not supply a universal explanation for those outcomes.

# 3 Experimental Methodology and Platforms

## 3.1 Frozen systems and workloads

The cross-platform hardware synthesis is retrospective: campaign-level primary outcomes were frozen independently. Later path controls and mechanism analyses retain their stated secondary, mechanism-supported, or diagnostic scope; they do not form a jointly preregistered cross-platform experiment.

Table 1 summarizes the completed hardware campaigns. The separately frozen runtime protocols and their distinct M4 estimator appear in Section 7. Final comparisons process one request at a time (sequence batch one). The final target is Qwen3-8B Q8_0; the native draft is Qwen3-0.6B Q8_0. The 358H heterogeneous adapter instead uses the qualified symmetric INT4 OpenVINO draft. We retain this quantization difference rather than describing the stacks as identical. Earlier M4 MLX-LM precision/correctness failures belong to qualification history and are distinct from the final llama.cpp Metal path.

The M4 representative sample freezes 24 previously unseen Spec-Bench prompts (Xia et al., 2024), with four from each of six workload groups and prior diagnostic IDs excluded. It uses three randomized paired repetitions. A separate preregistration freezes 30 HumanEval IDs (Chen et al., 2021) before prompt download or inspection; fixed $k = 2$ is the primary condition. The registered confidence-threshold conditions remain secondary. The subsequent HumanEval comparisons retain the frozen code-workload scope, three paired repetitions, greedy decoding, and a 128-token cap. They do not execute generated code.

The B570 mechanism follow-up separates outcome-informed discovery from reserved validation. An exposure audit conservatively excludes any HumanEval task with a recorded B570 association; 31 of 164 tasks are excluded, leaving 133 eligible. A deterministic rule selects 24 reserved tasks with no known exposure in the audited records. On 12 exposed discovery tasks, a frozen H1/H2 decision tree selects H1 only after target-only greedy micro-batch one matches all 36 serialized sequences. The hypothesis is committed before any reserved inference. All 72 complete reserved predictions are committed before the live serialized outcomes; success requires 72/72 direct-ID sequence matches, with no post-outcome predictor change. "No known exposure" describes the audited scope, not proof about unavailable records. The appendix gives the chronology and selection details.

Primary timing compares matched decode regions, excluding model loading and initial prompt prefill. In the native campaigns we use the frozen runtime decode counters; 358H explicitly times the complete post-prefill speculative loop. Replaying the committed prefix *inside* that loop remains charged to drafting. Thus the measurements are conditional decode costs, not whole-request latency including startup. Target-forward controls have their own narrower timing boundary. Different runtime versions, quantization, and device paths prevent a hardware-only causal comparison.

## 3.2 Statistical estimators and uncertainty

For each hardware-study prompt we reduce its three paired baseline/speculative ratios, then report the geometric mean across prompts. M4 and RTX use a within-prompt geometric mean, as does the B570 diagnostic analysis; 358H uses a within-prompt median. Intervals are 95% prompt bootstraps. M4 HumanEval uses 100,000 resamples; the other reported campaigns use 10,000. Frozen seeds and endpoint conventions are given in the separate appendix. We preserve campaign-specific estimators and finite-bootstrap input order rather than harmonizing them after observing outcomes.

These intervals quantify prompt sampling conditional on the campaign. They do not encompass all session, temperature, or runtime-state uncertainty. Randomized adjacent pairing reduces drift but does not remove it. We therefore keep the representative and code-conditioned M4 studies separate, including their differing session conditions.

## 3.3 Provenance and energy boundaries

Frozen definitions, processed summaries, scripts, and archived evidence anchor every quantitative claim in an internal ledger. Equality counts are interpreted using the comparator distinction in Section 2, not as uniform direct-ID evidence. Historical Windows raw per-run provenance is weaker, although committed pair tables and mechanism captures support recomputation. The new RTX/B570 qualification studies retain direct-ID traces and independently replayable archives; their common protocol family and distinct settings are described in Section 5.1. The 358H reconstructed draft source is `STRONGLY_SUPPORTED`; exact historical upstream identity is not proven. The header-only blocked-preflight derivative is excluded from final analysis.

Energy is interpreted only within its qualified sensor boundary. M4 measures package aggregate excluding separate DRAM; B570 measures adapter board energy. RTX board and CPU-package domains are not a common whole-system boundary, and its serialized HumanEval follow-up lacks an admissible formal energy comparison. No end-to-end Phase-3 energy claim is made for 358H. We do not rank platform energy efficiency.

*Table 1.* Frozen platform and runtime scope. All final target models are Qwen3-8B Q8_0. Greedy execution is compared with matched target-only references; historical M4/RTX/B570 use re-tokenized text, while 358H uses direct IDs. Hardware and software co-vary.

| System | Runtime / target backend | Draft model / placement | Evidence scope |
|---|---|---|---|
| MacBook Air M4 | llama.cpp b10809 / Metal | Qwen3-0.6B Q8_0 / Metal | Representative 24Q and separate HumanEval30 |
| Core Ultra 9 275HX + RTX 5070 Ti Laptop | llama.cpp b10839 / CUDA | Qwen3-0.6B Q8_0 / CPU or same GPU | Blocked fast path; text-path-qualified controls |
| Core i5-13400 + Arc B570 | llama.cpp b10839 / Vulkan0 | Qwen3-0.6B Q8_0 / B570 | Formal gate failure; later diagnostics |
| Core Ultra X7 358H + Arc B390 | llama.cpp b10839 / Vulkan0; OpenVINO draft | Qwen3-0.6B symmetric INT4 / CPU or NPU | Closed Phase 3; direct stateful decomposition |

*Table 2.* Headline decode outcomes. Speedup is baseline/speculative time, aggregated by the campaign's frozen prompt estimator. Brackets are 95% prompt-bootstrap CIs. Matching pairs and faster prompts are separate metrics. M4/RTX/B570 count re-tokenized-text agreement; 358H counts direct-ID agreement. B570 timing is diagnostic, despite a reported CI.

| Path / workload | Speedup [95% CI] | Matching pairs | Faster prompts | Class |
|---|---|---|---|---|
| M4 representative 24Q | 0.842 [0.809, 0.877] | 72/72 | 1/24 | Primary |
| M4 HumanEval fixed-$k = 2$ | 1.387 [1.351, 1.422] | 90/90 | 30/30 | Primary |
| CPU to RTX 5070 Ti Laptop | 0.404 [0.390, 0.418] | 90/90 | 0/30 | Second. |
| RTX 5070 Ti Laptop to itself | 0.650 [0.629, 0.673] | 90/90 | 0/30 | Second. |
| B570 serialized fallback | 0.3087 [0.2927, 0.3246] | 60/90 | 0/30 | Diag. |
| 358H CPU draft | 0.734 [0.693, 0.771] | 90/90 | 0/30 | Primary |
| 358H NPU draft | 0.352 [0.333, 0.369] | 90/90 | 0/30 | Primary |

## 4 Gains Do Not Translate Uniformly

Table 2 preserves historical timing and its recorded comparator: M4/RTX/B570 use the historical text-path comparator. The fresh direct-ID studies in Section 5.1 and Figure 3 measure qualification separately; they do not retrospectively certify these timings as strict-ID performance. A blocked path remains diagnostic.

### 4.1 M4: outcomes in two separate studies

The representative frozen M4 sample produces 0.842× speedup (95% CI [0.809, 0.877]), with 72/72 text-path-equal pairs and only 1/24 prompts faster. The separately preregistered HumanEval fixed-$k = 2$ study produces 1.387× [1.351, 1.422], with 90/90 text-path-equal pairs and 30/30 prompts faster. Figure 1 retains both; the positive code-conditioned result does not replace or pool with the negative representative result.

The same M4 supports both profitable and unprofitable observed regimes that pass the historical text-path comparator. Profitability is therefore not a simple hardware capability property in these studies. These outcomes do not establish general M4 profitability or isolate unified memory as a cause of the profitable result. These are workload-conditioned observed regimes, not a controlled estimate of workload causality: workloads and session conditions differ. The earlier discovery effect size is not promoted as a confirmatory estimate. Historical MLX failures also do not negate the text-path-qualified historical Q8_0 Metal result.

For primary fixed-$k = 2$ HumanEval, the package-efficiency ratio (baseline/speculative J per output token) is 1.013× [0.990, 1.034]. The interval includes parity; a package-energy improvement is not established. The registered $p_{\min} = 0.9$ condition has secondary energy evidence, discussed separately in the appendix. The preregistered corrected synthetic ceiling was not executed. We retain that deviation rather than treating the earlier synthetic diagnostic as its replacement.

### 4.2 Across stacks: negative and blocked outcomes

On RTX 5070 Ti Laptop, the serialized CPU-to-RTX control reaches 0.404× [0.390, 0.418], while the same-GPU draft control reaches 0.650× [0.629, 0.673]. Both pass 90/90 text-path comparisons and accelerate zero of 30 prompts. The higher speed of same-GPU placement shows that placement and associated integration costs matter, but its continued loss rules out transfer as the sole explanation.

On Arc B570, neither the fast path nor the serialized fallback supports a workload-global admissible optimization result.

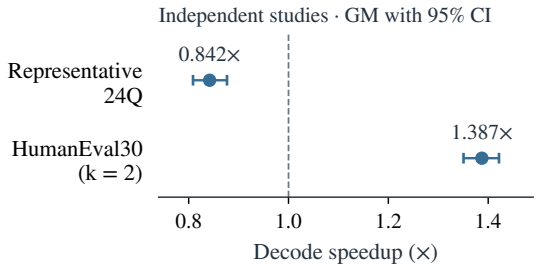



*Figure 1.* **Opposite profitability regimes on the same tested M4 stack**. Representative 24Q yields 0.842× [0.809, 0.877], while the separately preregistered HumanEval30 fixed-$k = 2$ study yields 1.387× [1.351, 1.422]. Points are campaign prompt geometric means of paired decode speedups; intervals are campaign-specific 95% prompt-bootstrap intervals. The dashed line marks parity. Both pass the recorded re-tokenized-text comparator (72/72 and 90/90 pairs); these are not direct generated-ID comparisons or functional HumanEval pass@$k$. These are separate studies, not a paired between-workload comparison; differing workloads and session conditions do not isolate workload-only causality.

The latter's diagnostic speedup is 0.3087× [0.2927, 0.3246], with zero of 30 prompts faster. That interval describes timing on a partially mismatching path; its precision does not turn it into an admissible speed claim. The formal failure and stop rule remain the platform result. Board-energy observations likewise remain diagnostic and boundary-limited.

On Core Ultra X7 358H, the closed Phase-3 heterogeneous runner is exact on both CPU and NPU drafting: 90/90 pairs per path, with zero of 30 prompts faster. CPU achieves 0.734× [0.693, 0.771]; NPU achieves 0.352× [0.333, 0.369]. These exact negative outcomes coexist with favorable isolated draft measurements. Section 5 investigates which measured boundaries account for the failed local promise.

# 5 From Local Opportunity to Realized Gain

The observed outcomes trace two tested boundaries: execution-path qualification and stateful realization. Figure 2 locates them. The B570 follow-up then tests a specific explanation for residual serialized divergence.

## 5.1 Execution-path qualification

**Historical physical opportunity.** Figure 3A preserves eleven historical target-forward cells. The M4 Metal micro ratios are $V_2^{\text{micro}} \approx 1.02$–$1.08$; RTX batching is near one baseline step while serialization costs more; B570 fast cells span 1.24–1.64 and serialized cells approximately 1.983–1.992. These proxies omit drafting and integration, cannot replace complete $R_{\text{target}}$, and do not explain the separate-session M4 contrast.

**Prospective direct-ID qualification.** Two frozen studies share 30 HumanEval tasks, three repetitions, greedy $k = 2$, and the Qwen3-8B/0.6B Q8_0 pair. Each completes 270 executions and 180 comparisons. The gate requires 90/90 complete direct-ID sequences with no dropped mismatch. Different runtime/front-end, context, and reference settings preclude controlled hardware comparison or fresh performance claims.

RTX batching matches 75/90 pairs and serialization 90/90, qualifying only the latter. B570 fast matches 60/90 and serialization 78/90; both remain blocked. All conditions are repetition-stable on 30 prompts, which does not imply reference equality. Figure 3B separates these endpoints from historical cost.

**Serialization changes qualification but is not a certificate.** The prospective direct-ID results do not replace historical B570 text-path counts or certify historical RTX timing; the older RTX controls remain secondary and B570 timing diagnostic. The earlier passing small B570 probe was limited to its subset.

## 5.2 Controlled B570 mechanism isolation

The v0.7 B570 residual could reflect reference, front-end, context, or target-partition differences. A target-only harness first reproduces 90/90 reference sequences. On 12 outcome-exposed discovery prompts, target-only greedy execution with $n_{\text{ubatch}} = 1$ matches 36/36 serialized speculative sequences. The frozen H1/H2 decision tree selects H1 (target-partition sufficiency); H2 is not run. Discovery is exploratory.

An exposure audit conservatively excludes 31 tasks, then deterministically selects 24 reserved HumanEval prompts from 133 eligible tasks. H1 is committed before reserved inference; all 72 complete target-only predictions are committed before any live serialized outcome. The live runs match 72/72 predicted sequences, 24/24 prompt classifications, and 12/12 divergent repetition-pair first-divergence indices across four prompts. Predicted and actual reference equality is 60/72, so qualification remains *blocked*. No parameter or prediction changes after their respective freezes; no selective reruns occur. The appendix gives task IDs, divergence tokens, and audit scope.

Target-only micro-batch one thus reproduces the tested serialized trajectory without live draft computation. It isolates a sufficient target execution partition, not a unique kernel, universal rule, or RTX mechanism. The 60/72 reserved endpoint is distinct from the earlier 78/90 direct-ID study.

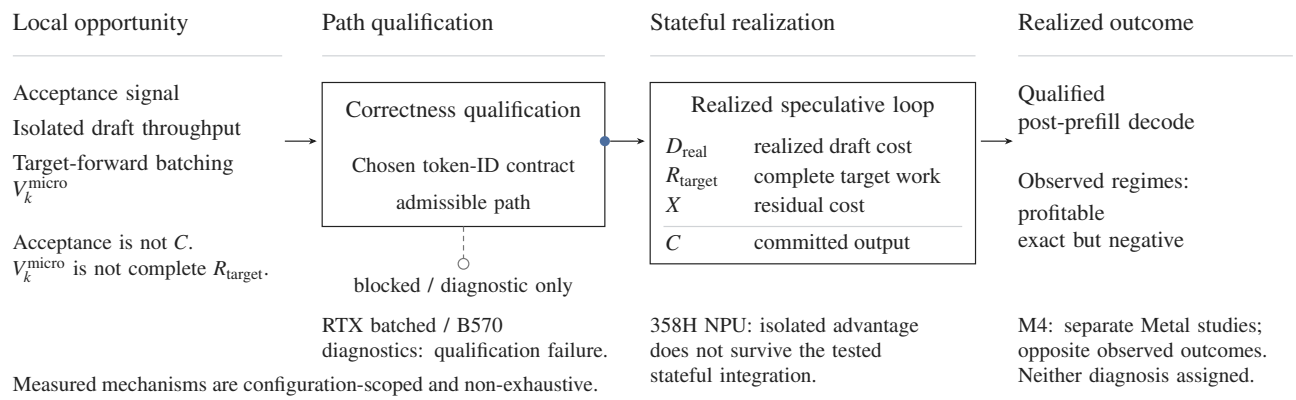



*Figure 2.* **From local opportunity to integrated post-prefill decode outcomes.** Local acceptance, isolated drafting, and target-forward batching do not by themselves establish integrated gains. Arrows mark evaluation boundaries, not a guarantee of gain. The output requirement excludes the tested RTX 5070 Ti Laptop batched path; B570 serialization improves agreement without qualifying the full workload. Blocked timings remain diagnostic. For the qualified 358H NPU path, full-prefix replay and proposal share one synchronous OpenVINO call, and the isolated draft advantage does not survive the tested stateful integration. $D_{\text{real}}$, $R_{\text{target}}$, and $X$ normalize measured draft, complete target, and residual costs by matched target-only token cost; $C$ counts all committed output, including correction/bonus tokens. M4's separate frozen studies yield opposite profitability regimes with text-path agreement. Historical M4/RTX/B570 equality labels refer to that comparator, not direct-ID certification. Workload and session conditions differ, and neither diagnosis is assigned to the contrast. The measured mechanisms are configuration-scoped and non-exhaustive; timing excludes loading and initial prompt prefill.

**Separate RTX numerical diagnostic.** A frozen CUDA reproducer captures repeat-stable HumanEval/106 argmax divergence, with a same-commit diagnostic score-margin reversal (appendix). This historical batch-shape observation (Yuan et al., 2025; Gond et al., 2026) does not identify a kernel, replace formal measurements, or explain B570.

### 5.3 Stateful realization

On Core Ultra X7 358H, isolated NPU draft throughput exceeds CPU at tested contexts, yet both qualified integrated paths are unprofitable and NPU is slower. Direct round records show complete mean committed progress $C = 2.4631$ and $2.4584$, respectively, rather than the older accepted-draft counter.

Complete target work, including persistent state, verification, rollback, and correction/bonus decoding, accounts for 87.34% of CPU-path time. On NPU, the synchronous OpenVINO call replays the committed prefix and proposes tokens, accounting for 58.35%. Replay and proposal are inseparable. Residual orchestration is 0.04% and 0.02%, respectively.

Figure 4 separates isolated throughput from realized cost. The result concerns this qualified interface, not NPU compute capability; Phase-3 timing does not estimate a persistence benefit.

Arithmetic totals reconstruct Equation 2: CPU $(D_{\text{real}}, R_{\text{target}}, X) = (0.4304, 2.9767, 0.00124)$ yields $0.7227\times$; NPU $(4.1251, 2.9435, 0.00130)$ yields $0.3477\times$. These aggregate ratios differ from Table 2's prompt-level geometric means.

The Phase-2 eligibility screen combined isolated drafting and a target-forward proxy; Phase 3 measures complete target and draft-interface cost instead. There is no Phase-3 end-to-end energy claim; source reconstruction remains `STRONGLY_SUPPORTED`.

## 6 From Measured Constraints to Runtime Decisions

Two measured boundaries constrain usable speculation. A cheap verification path can fail the output contract, and a fast isolated draft device can incur additional work through its state interface. The B570 held-out prediction shows that target execution partition alone can reproduce the observed serialized output trajectory without live drafting on the tested stack. Complete target-side work can dominate even after draft cost is lowered, as on 358H CPU drafting.

RTX serialization crosses the 90/90 gate, while B570 serialization improves agreement without crossing it. First qualify the intended path against its output contract. If it fails, isolate the target partition before assigning the failure to live drafting or device placement. Only then use complete realized cost to assess profitability on a qualified path. This evidence-derived sequence is bounded by the tested configurations. The separately frozen runtime studies on different stacks are not an intervention/ablation chain.

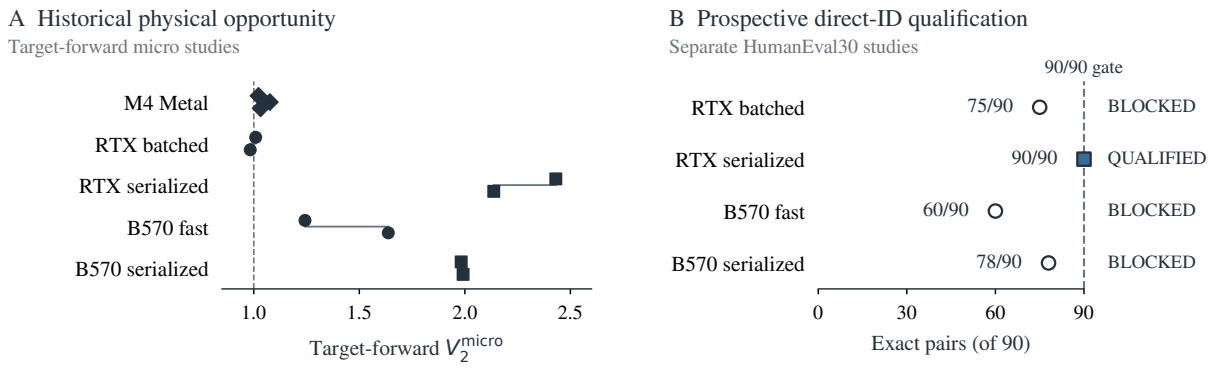



*Figure 3.* **Verification cost and direct-ID qualification are separate execution-path properties.** A: historical target-forward $V_2^{\mathrm{micro}}$ cells from separate mechanism studies; shapes encode M4 reference (diamond), fast/batched (circle), and serialized (square) paths, with no qualification encoding. Segments span measured contexts, not uncertainty. This proxy is not complete $R_{\mathrm{target}}$ or end-to-end speedup. B: new prospectively frozen HumanEval30 direct-ID studies; only RTX serialization reaches the preregistered 90/90 gate. B570 serialization improves agreement but remains blocked. Open circles mean BLOCKED; the filled square means QUALIFIED. The panels are not a same-run cost-versus-correctness experiment. Results are path/runtime scoped; contexts and references differ. Appendix tables retain the historical context lookup and new qualification counts separately.

## 7 Qualification and Realized-Cost Runtime Control

We examine qualification and continuation in separately frozen runtime studies. The 358H candidate stops at qualification. On M4, an available exact path permits a prospective test of a simple cost rule: Guard remains faster than target-only, but fixed speculation has the higher aggregate speedup point estimate. These are complementary tests of the decisions in Section 6, not two successful optimization interventions.

### 7.1 An intervention stopped by qualification

The 358H component measurements motivated persistent draft state and a wider target execution setting. The implementation ran the target and draft backends in separate processes. Synchronous draft calls, including communication and state work, remained inside the target worker's post-prefill timer. The executable implementation still had to pass correctness qualification. CPU persistent-state restore/reconcile correctness could not be established under the tested representation; the Optimum NPU state-capable path was unavailable with the frozen dynamic-shape IR. These are implementation boundaries, not evidence that the devices cannot retain state.

The target setting `n_ubatch=2` (UB2) passed the small gate but failed full qualification. Correctness-only follow-up controls on Qwen3-8B Q8_0, llama.cpp b10839, and Vulkan0 reproduced different greedy tokens under UB1 and UB2 even with identical exact prefix tokens. Request-order and separate-process controls reproduced the target-only divergence. Replaying frozen proposals also reproduced the speculative divergence without live NPU draft inference. The latter depended on earlier target execution history; the controls found no position drift or specific rollback-boundary bug. The responsible floating-point or backend operation remains unidentified.

Under this frozen tested target stack, UB2 did not preserve the strict exact-output contract. The formal registry retained only the target reference, so the runtime study had no qualified intervention pair to evaluate. No formal R2 performance run occurred, and the follow-up diagnostics are invalid for performance claims. Qualification excluded an intervention motivated by the measured bottlenecks before its speed was evaluated.

### 7.2 A prospective cost guard on M4

The M4 experiment asks whether a frozen rule based on already-realized complete cost can control continuation while preserving exact output and profitable execution against target-only. It does not assume that adding control improves on fixed speculation. We retain the Qwen3-8B/0.6B Q8_0 pair on llama.cpp Metal, fixed $k = 2$, greedy generation, and a 128-token cap. Guard begins on the same speculative path as Always-Spec and can fall back permanently to target-only within a request. It changes neither proposal width nor device placement.

At each eligible completed round, Guard divides all elapsed charged generation cost by committed output progress $C$.

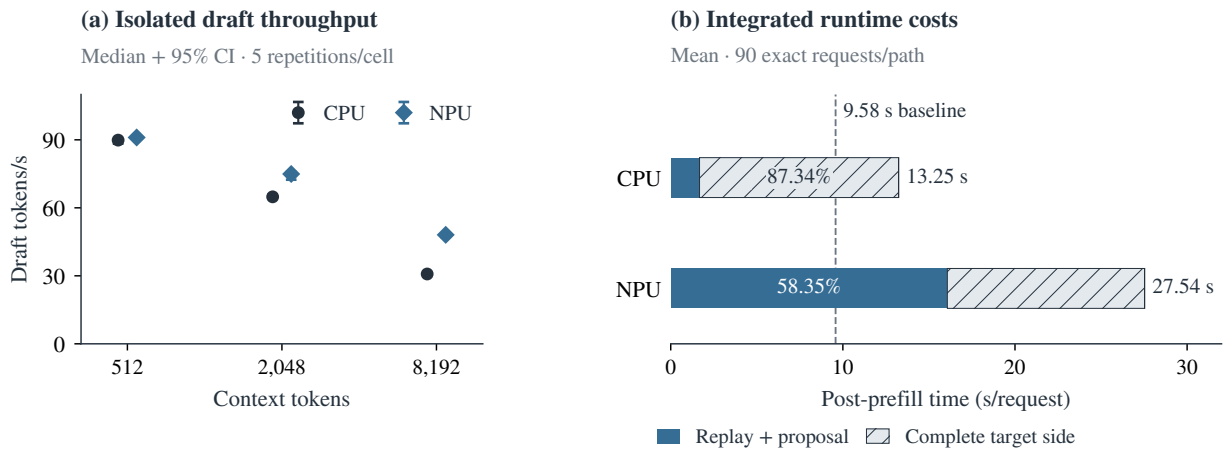



*Figure 4.* **Isolated draft advantage does not survive the tested stateful integration**. (A) Phase-1 median steady-state draft throughput and 95% repetition-bootstrap intervals, five repetitions per cell; the NPU median advantage grows across the tested contexts. (B) Phase-3 arithmetic mean post-prefill seconds per request over 90 observations per path, all exact. CPU and NPU totals are 13.25 s and 27.54 s against a 9.58 s mean baseline. The CPU path is target-side dominated (87.34%); the NPU path spends 58.35% in the measured replay-plus-proposal call. The 0.734× and 0.352× headlines use the separate prompt-GM estimator and are not ratios of the arithmetic mean bars. Replay and proposal share one timed OpenVINO call; replay-only time is not measured. Initial prompt prefill is excluded. This diagnoses the tested runtime interface, not intrinsic NPU capability or a measured benefit from a state-persistent intervention. CPU replay plus proposal is 12.63%; NPU complete target work is 41.63%. Residuals are 4.81 ms (0.04%) and 5.06 ms (0.02%). Prompt-GM speedup 95% CIs are [0.693, 0.771] and [0.333, 0.369], respectively. The vertical dashed line is the shared arithmetic-mean baseline. Isolated throughput excludes the full-prefix state cost; device marks are horizontally offset for readability.

This count includes the initial target seed and committed correction or bonus tokens, but excludes rejected proposals. Guard continues if this cumulative cost per token is at most $(1 - m)R_{\text{ref}}$, a target-only reference frozen from calibration; otherwise it falls back. If a decision is reached with zero committed progress, Guard also falls back. Before each control decision, invalid candidate suffix state is removed from the target and draft contexts, preserving committed history for target-only continuation. The enclosing post-prefill clock includes generation-loop initialization, seed-token sampling, drafting, target work, synchronization, state reconciliation, control, and transition costs. Model loading, sampler/speculator setup, initial prompt prefill, and the speculative begin hook precede the timer; the result is not whole-request latency.

The reference is the median across calibration prompts of each prompt's median target latency divided by its median output-token count. The preregistered candidate grid varies the first decision round and margin. Selection minimizes calibration geometric-mean latency relative to the offline minimum of target-only and Always-Spec among exact candidates, with frozen tie-breaks. The selected rule uses margin 0 and first decides after 8 completed speculative rounds; it then decides after each later round while speculation continues. A request ending earlier makes no decision. The online policy receives neither task/family labels nor future output length, current-request target-only latency, or oracle outcomes.

### 7.3 Held-out evaluation

Historical M4 outcomes informed the study design. Prospective here means that the policy was frozen before the reserved prompt outcomes were measured, not that the design was uninformed by earlier campaigns. IDs marked as previously executed or outcome-exposed in the frozen inventory were excluded before deterministic hash-ranked selection. Disjoint splits contain 40 calibration prompts, 16 qualification prompts, and 60 held-out prompts, with equal HumanEval and Spec-Bench counts within each split. The policy and reference were frozen before qualification and formal measurement; the appendix documents the audited recovery of a freeze-hash metadata defect. Both Always-Spec and Guard passed 48/48 direct-token comparisons in qualification.

The formal study runs 3 repetitions of target-only, Always-Spec, and Guard on each held-out prompt, randomizing condition order within prompt and repetition: 540 observations. Both speculative conditions are 180/180 direct-token exact, with zero mismatches. For each prompt and condi-

*Table 3.* M4 per-prompt median post-prefill latencies. Each speculative condition is 180/180 direct-token exact. The first two rows report speedups. The offline oracle uses the lower target-only or Always-Spec median and is unavailable online; its row reports latency regret. The separated Guard/Always-Spec contrast is reviewer-requested post-study paired secondary analysis; values above one mean Guard is slower.

| Latency ratio | GM | 95% prompt CI |
|---|---|---|
| Target / Guard | 1.3114 | [1.2487, 1.3787] |
| Target / Always-Spec | 1.3469 | [1.2867, 1.4088] |
| Guard / offline oracle | 1.0278 | [1.0137, 1.0446] |
| *Post-study paired secondary latency ratio* | | |
| Guard / Always-Spec | 1.0271 | [1.0130, 1.0436] |

tion, analysis takes the median latency across repetitions, then the geometric mean of prompt latency ratios. This ratio-of-medians estimator differs from the historical M4 hardware estimator. Confidence intervals use 10,000 prompt bootstrap resamples with seed 20260924; repetitions are not independent sampling units.

Guard is profitable against target-only on this held-out workload (Table 3). Fixed speculation has the higher aggregate speedup point estimate. A reviewer-requested post-study paired secondary analysis of the frozen prompt medians gives Guard/Always-Spec latency 1.0271 [1.0130, 1.0436]. The interval lies above parity: Guard had higher latency on this comparison. By prompt median, Guard is faster on 24 prompts, Always-Spec on 36, with 0 ties. Guard falls back in 30.6% of its 180 request executions; this measures control activity, not the fraction of beneficial decisions. Guard's geometric-mean latency relative to the offline oracle is 1.0278×. That oracle chooses the lower target-only or Always-Spec median after measurement and was unavailable to the runtime. It is a descriptive hindsight comparator, not an achievable-policy guarantee.

The result demonstrates exact, profitable prospective execution of a complete-cost continuation rule. Neither fallback activity nor the oracle ratio establishes an incremental benefit of control, risk reduction, or threshold optimality. The experiment supplies no evidence of policy transfer to another stack. Small individual Spec-Bench strata do not support stable subgroup estimates. No runtime energy claim is made. The separate appendix documents the archived recomputation and audited freeze-hash metadata exception.

## 8 Related Work

**Performance and cost accounting.** Speculative decoding amortizes target computation while preserving a target sampling distribution under its algorithmic assumptions (Leviathan et al., 2023). Liu et al. (2026b) study verification dominance and workload/batch effects in vLLM; Kim et al. (2026) control runtime policies and account for committed work and conditional execution cost. Lyu et al. (2026) derive edge–cloud profitability conditions. On consumer Apple silicon, Chordiya (2026) reports positive and negative regimes across five backend configurations, with three decelerating, and diagnose ineffective batch-parallel verification in a quantized Metal configuration. The study also tests distribution agreement and greedy equality, and reports target-only batch-shape argmax changes. Slowdown, latency-gap profitability, numerical path dependence, and correctness gating are therefore close prior art. Our distinction is prospective direct generated-ID qualification on RTX/B570, followed by a B570 target-partition hypothesis frozen before reserved inference and a 72/72 complete serialized-sequence prediction. We separate physical opportunity from admissibility under the output contract.

**Heterogeneous execution.** Dovetail places drafting on a GPU and verification on a CPU (Zhang et al., 2025); DuoDecoding uses hardware-aware CPU/GPU draft budgets (Lv et al., 2025); Mirror-SD considers complementary GPU/NPU pipelines (Bhendawade et al., 2025). Ruiz y Mesa et al. (2026) combine an analytical cost model with compiler-oriented partitioning, predicting profitable CPU/GPU operating points validated on Cortex-A/Mali hardware. Lever jointly optimizes drafting, verification, and CPU/NPU execution for a DRAM-resident draft and flash-backed smartphone target (Wang et al., 2026). These systems establish heterogeneous placement and predictive performance modeling. Our B570 predictor instead concerns measured output trajectories and qualification. Our 358H result measures loss of an isolated draft-device advantage through one realized state interface; it does not contradict successful heterogeneous co-design such as Lever.

**Adaptive and contextual speculation.** TurboSpec profiles execution and adjusts or disables speculation for serving goodput (Liu et al., 2025). BanditSpec selects configurations online with stopping-time regret analysis (Hou et al., 2025); our oracle ratio remains descriptive. ACL HeteroSpec estimates verification complexity from entropy and adapts depth and pruning to contextual heterogeneity (Liu et al., 2026a). Context-adaptive verification allocation is established. Our B570 predictor is a mechanism diagnostic for qualification, not an adaptive decoding policy. M4 supplies a runtime endpoint without incremental Guard benefit or serving-controller comparison.

**Correctness and numerical reproducibility.** Zhang et al. (2026) distinguish ragged-batch synchronization errors from residual numerical mismatches, revising their earlier *Batch Speculative Decoding Done Right* preprint. Yuan et al. (2025) study numerical output variation across system

configurations; LLM-42 enforces deterministic inference through verified speculation (Gond et al., 2026). Numerical path dependence and deterministic speculative inference are established. Our contribution concerns their systems consequence for execution-path admissibility and a prospectively frozen target-partition prediction, rather than a new numerical phenomenon or synchronization diagnosis.

**State and memory reuse.** Yuan et al. (2026) maintain incrementally updated compressed draft KV memory for long contexts. Such designs differ from our measured full-prefix replay interface, but establish state reuse as prior art. The 358H evidence identifies realized replay-plus-proposal cost; its later intervention stops at qualification. Neither it nor the M4 control result measures a persistence benefit.

## 9 LIMITATIONS

The four consumer systems use a narrow Qwen3 model family, with hardware, operating systems, runtime versions, backends, and some quantization choices co-varying. The cases establish specific outcomes and within-platform mechanisms, not an architectural ranking, a population failure rate, or controlled portability. The two diagnoses do not causally explain the separate-session M4 contrast. Serving concurrency, production load, and broader model families are outside the measured scope.

The strict output contract is conditional on the frozen reference; another quality objective could admit other paths. Historical M4, RTX, and B570 equality counts use re-tokenized CLI text, with campaign-specific parsing. They cannot establish direct generated-ID equality; the historical labels and counts remain evidence of their recorded comparator only. Two fresh RTX/B570 studies supply direct generated-ID evidence without retroactively certifying historical timing. They cover one model pair, HumanEval30, fixed $k = 2$, and three repetitions; differing runtime/reference/context settings prevent estimating a causal hardware difference. The fresh RTX driver is 616.92 versus historical 616.56. B570 launcher recovery changes deployment, not frozen study semantics. The follow-up H1 predictor covers one B570 platform, model pair, runtime/build, greedy $k = 2$, HumanEval workload, and 24 reserved prompts. "No known B570 exposure" means no record in the audited scope, not proof that a task was never run. Target partition is sufficient to reproduce these trajectories, but the study neither identifies a specific kernel nor proves an architecture-wide cause, a CUDA/B570 common mechanism, or a universal micro-batch rule. The predictor is explanatory evidence, not a production certificate. Prompt counts do not establish backend-wide behavior. HumanEval functional execution and pass@$k$ were not evaluated. CUDA/B570 and the 358H runtime qualification findings concern their tested paths, not general backend unsuitability. Replay and proposal share one timed region, so their individual causal costs and a persistence benefit are not established. The 358H intervention did not reach formal performance evaluation.

Campaign estimators and bootstrap conventions differ. Prompt intervals do not cover all session/thermal uncertainty. Timing excludes initial prefill and loading and is not whole-request user latency. Energy domains differ, the corrected M4 synthetic-ceiling preregistration remains unexecuted, and no common energy ranking or end-to-end 358H Phase-3 energy claim is supported.

The runtime result covers one M4 model/runtime pair, 60 held-out prompts, fixed $k = 2$, and one-way fallback. The paired Guard/Always-Spec contrast is post-study secondary analysis, not the registered primary endpoint; its prompt-bootstrap uncertainty excludes session and recalibration variation. Guard had higher latency on this frozen comparison. Policy transfer, stable small-stratum conclusions, and runtime energy benefit remain unestablished. Historical outcomes informed the design; policy freezing protects the reserved prompt evaluation, not ignorance of earlier platform results.

Historical Windows raw per-run provenance is weaker than in the other campaigns; the new direct-ID studies retain independently validated raw evidence. The 358H draft-source reconstruction is `STRONGLY_SUPPORTED`, not proven historical identity, and its runtime diagnostic is available as a compact record without local raw-trace replay. The M4 freeze-hash metadata defect was independently reconciled against the actual freeze, formal-start commit, and archived observations. The erroneous original markers and pipeline stop remain intact; the appendix explains this recovery rather than treating it as uninterrupted pipeline validation.

## 10 CONCLUSION

Frozen direct-ID studies show that serialization qualifies the tested RTX path but leaves B570 blocked. A B570 follow-up predicts all 72 reserved serialized trajectories from target-only micro-batch-one execution, locating a sufficient target execution partition on the tested stack. On 358H, the state interface consumes an isolated draft-device advantage; its later intervention stops at qualification. The M4 guard is exact and profitable against target-only but slower than fixed speculation in the paired secondary comparison. Qualify the intended path, isolate its target partition if it fails, then judge profitability from complete realized cost.

## REFERENCES


Bhendawade, N., Nishu, K., Kundu, A., Bartels, C., Cho, M., and Belousova, I. Mirror speculative decoding: Breaking the serial barrier in LLM inference. *arXiv preprint arXiv:2510.13161v2*, 2025. URL https://arxiv.org/abs/2510.13161v2.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Petroski Such, F., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Hebgen Guss, W., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374v2*, 2021. URL https://arxiv.org/abs/2107.03374v2.

Chordiya, P. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware. *arXiv preprint arXiv:2607.17283*, 2026. doi: 10.48550/arXiv.2607.17283. URL https://arxiv.org/abs/2607.17283.

Gond, R., Kamath, A. K., Ramjee, R., and Panwar, A. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint arXiv:2601.17768v2*, 2026. URL https://arxiv.org/abs/2601.17768v2.

Hou, Y., Zhang, F., Du, C., Zhang, X., Pan, J., Pang, T., Du, C., Tan, V., and Yang, Z. BanditSpec: Adaptive speculative decoding via bandit algorithms. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 24045–24079. PMLR, 2025. URL https://proceedings.mlr.press/v267/hou25h.html.

Kim, S., Kim, J., Cho, Y., and Seo, J. Runtime-controlled evaluation of speculative decoding methods. *Electronics*, 15(14):3179, 2026. doi: 10.3390/electronics15143179. URL https://www.mdpi.com/2079-9292/15/14/3179.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, S., Ye, Y., Zhu, Q., Cao, Z., and He, Y. HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12930–12947. Association for Computational Linguistics, July 2026a. doi: 10.18653/v1/2026.acl-long.589. URL https://aclanthology.org/2026.acl-long.589/.

Liu, X., Park, J., Hu, L., Kwon, W., Li, Z., Zhang, C., Du, K., Mo, X., You, K., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput. *arXiv preprint arXiv:2406.14066v3*, 2025. URL https://arxiv.org/abs/2406.14066v3.

Liu, X., Yu, J., Park, J., Stoica, I., and Cheung, A. Speculative decoding: Performance or illusion? In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1719–1741, 2026b. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html.

Lv, K., Guo, H., Guo, Q., and Qiu, X. DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting. *arXiv preprint arXiv:2503.00784*, 2025. URL https://arxiv.org/abs/2503.00784.

Lyu, Y., Irukulapati, B., and Champati, J. P. Speculation at a distance: Where edge-cloud speculative decoding actually pays off. *arXiv preprint arXiv:2606.25091v2*, 2026. URL https://arxiv.org/abs/2606.25091v2.

Ruiz y Mesa, A., Korol, G., Riesterer, M., Cardoso de Lima, J. P., and Castrillon, J. Compiler-assisted speculative sampling for accelerated LLM inference on heterogeneous edge devices. In *8th Workshop on Accelerated Machine Learning (AccML), co-located with HiPEAC*, January 2026. URL https://accml.dcs.gla.ac.uk/papers/2026/8th_AccML_paper_8.pdf.

Wang, T., Li, F., Sun, Y., Gao, W., and Ren, J. Lever: Speculative LLM inference on smartphones. *arXiv preprint arXiv:2605.16786*, 2026. doi: 10.48550/arXiv.2605.16786. URL https://arxiv.org/abs/2605.16786.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.456. URL https:

//aclanthology.org/2024.findings-acl.456/.

Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint arXiv:2506.09501v2*, 2025. URL https://arxiv.org/abs/2506.09501v2.

Yuan, T., Liao, C., and Wen, Z. Strong drafts need compact memories: Long-context speculative decoding with compressed KV cache. *arXiv preprint arXiv:2608.30252*, 2026. URL https://arxiv.org/abs/2608.30252. EMNLP 2026 Findings.

Zhang, L., Zhang, Z., Xu, B., Li, R., Tian, Z., Mei, S., and Li, D. Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference. *arXiv preprint arXiv:2412.18934v2*, 2025. URL https://arxiv.org/abs/2412.18934v2.

Zhang, R. H., Dey, S., Mishra, A., Wu, H., Li, B., and Zhang, R. Correctness forensics for batch speculative decoding: Diagnosing the ragged tensor problem. *arXiv preprint arXiv:2510.22876v4*, 2026. URL https://arxiv.org/abs/2510.22876v4. Accepted to Findings of EMNLP 2026.